

CSCE 4613/5613
Artificial Intelligence

Midterm

Reviews

Spring 2026

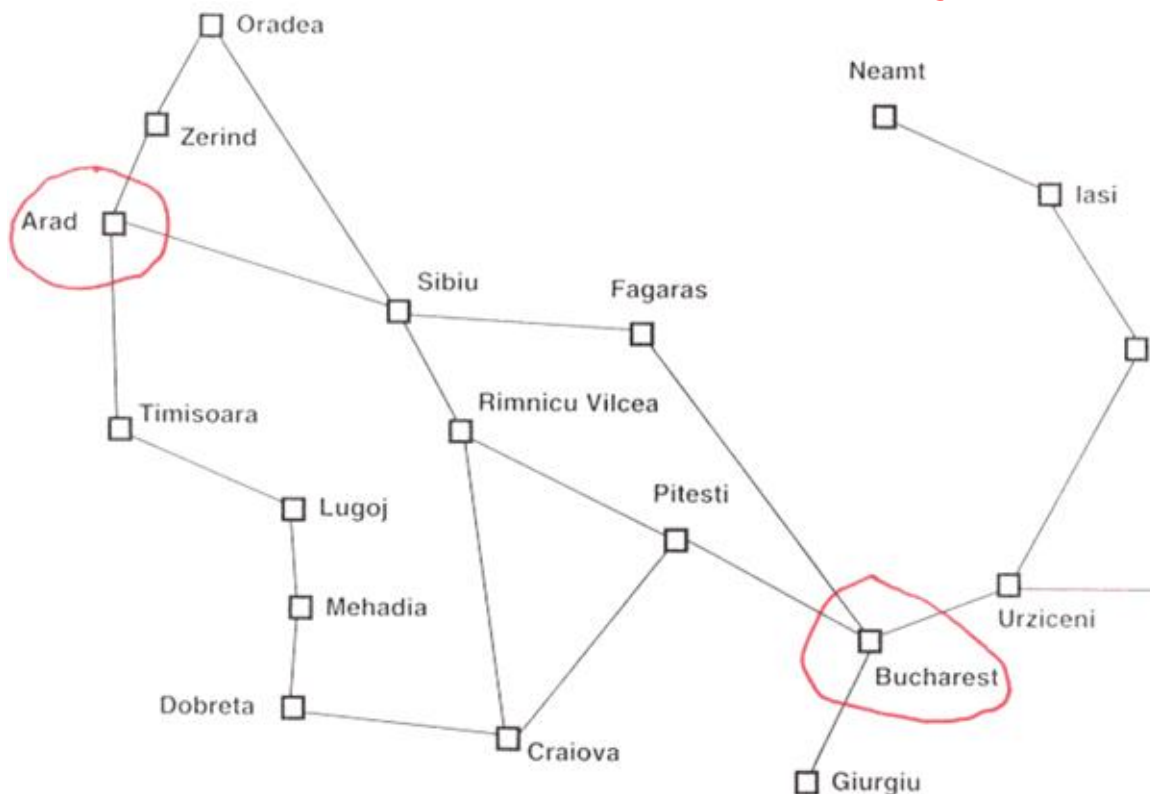
Prof. Khoa Luu
khoaluu@uark.edu

Midterm Exam

- Individual exam
- Closed book exam
- Allow one A-4 paper (2 slides) with any writing you want
- Coding, algorithm running, and theoretical questions
- Midterm exam time: 75 minutes

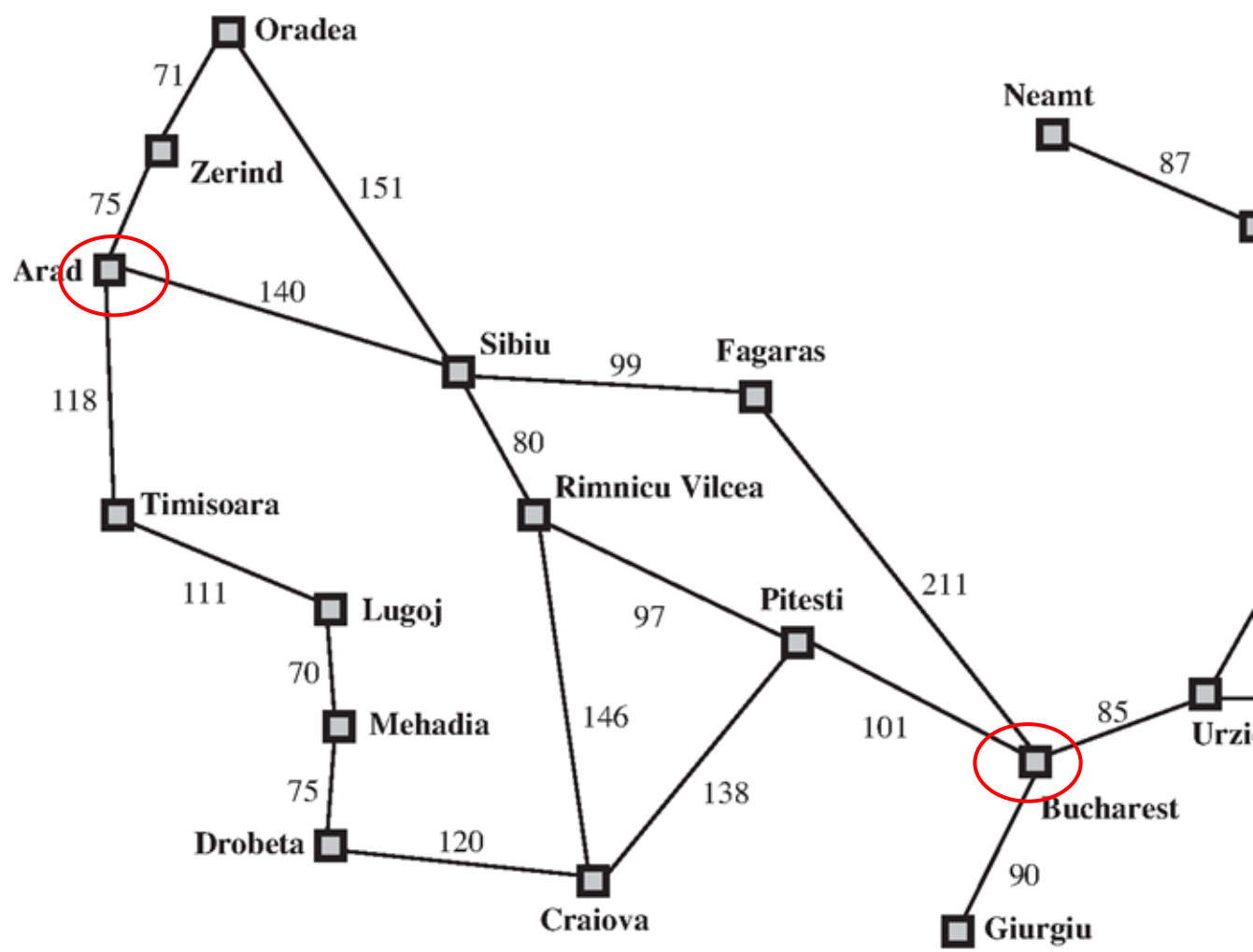
Search

- Goal-based agent
- In some applications we are interested in finding a path to the goal, in other applications we are interested in finding a path to the goal



TREE SEARCH

Best Path from Arad to Bucharest

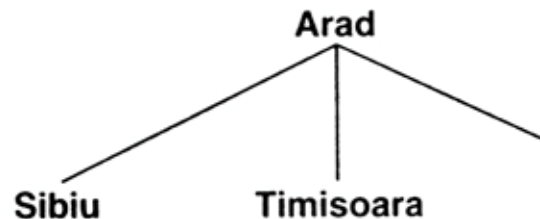


Partial search tree for route finding from Bucharest

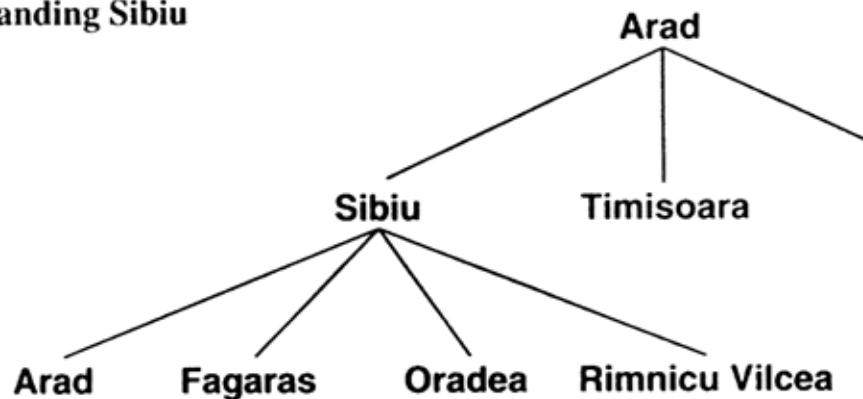
(a) The initial state

Arad

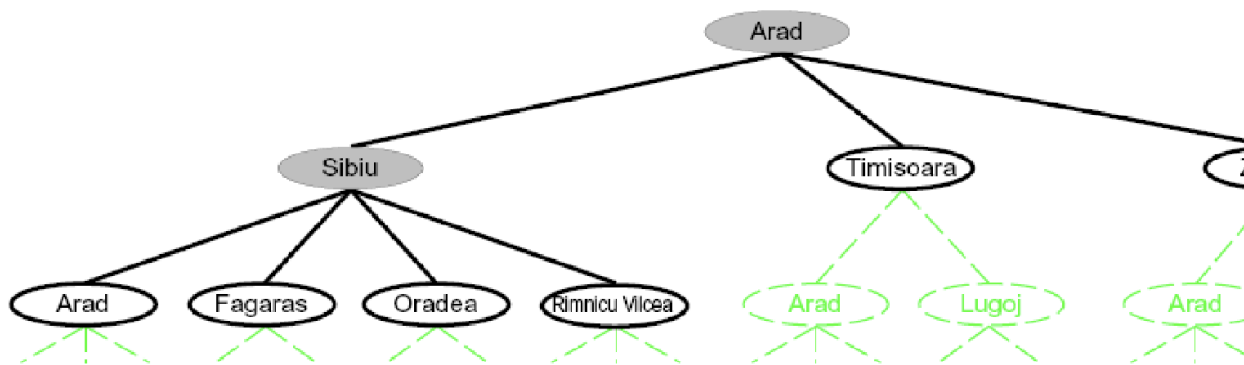
(b) After expanding Arad



(c) After expanding Sibiu



The General Search Algorithm



- Search:
 - Expand out potential plans (tree nodes)
 - Maintain a **fringe** of partial plans under co
 - Try to expand as few tree nodes as possible

The General Search Algorithm

```
function TREE-SEARCH( problem, strategy ) returns a solution, or failure
  initialize the search tree using the initial state of problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
  end
```

- Important ideas:
 - Fringe
 - the set of nodes that are to-be-visited
 - Expansion
 - the process of ‘visiting’ a node
 - Exploration strategy
 - how do we decide which fringe node to expand
- Main question: which fringe nodes to expand

Measures of Performance



Completeness

Guaranteed to find a solution when there is one?



Optimality

Finds an optimal solution?



Time

How long does it take to find a solution?

An aside: A bounded rationality perspective
Search cost is about time cost and space cost.
Domain cost = path cost
Total cost = domain cost + search cost

Uninformed vs. Informed Search



Uninformed

Can only generate successors
and distinguish goals from non-
goals



Informed

Strategies that know
non-goal is more precise than
another

Remarks on Uniform Cost Search (UCS)

- UCS is an extension of BFS to the weighted-graph case (UCS = BFS if all edges have the same cost).

Remarks on Uniform Cost Search (UCS)

- UCS is an extension of BFS to the weighted-graph case (UCS = BFS if all edges have the same cost).
- UCS is complete and optimal (assuming costs bounded away from zero).

Remarks on Uniform Cost Search (UCS)

- UCS is an extension of BFS to the weighted-graph case (UCS = BFS if all edges have the same cost).
- UCS is complete and optimal (assuming costs bounded away from zero).
- UCS is guided by path cost rather than path depth, so it may find a solution even if some edge costs are very small.

Remarks on Uniform Cost Search (UCS)

- UCS is an extension of BFS to the weighted-graph case (UCS = BFS if all edges have the same cost).
- UCS is complete and optimal (assuming costs bounded away from zero).
- UCS is guided by path cost rather than path depth, so it may find a solution even if some edge costs are very small.
- Worst-case time and space complexity $O(b^{W^*/\epsilon})$, where W^* is the cost of the optimal solution and ϵ is such that all edge weights are no smaller than ϵ .

Greedy (Best-First) Search

- UCS explores paths in all directions, with no bias towards

Greedy (Best-First) Search

- UCS explores paths in all directions, with no bias towards
- What if we try to get “closer” to the goal?

Greedy (Best-First) Search

- UCS explores paths in all directions, with no bias towards
- What if we try to get “closer” to the goal?
- We need a measure of distance to the goal. It would be the length of the shortest path... but this is exactly what we want to compute!

Greedy (Best-First) Search

- UCS explores paths in all directions, with no bias towards
- What if we try to get “closer” to the goal?
- We need a measure of distance to the goal. It would be the length of the shortest path... but this is exactly what we want to compute!
- We can estimate the distance to the goal through a heuristic $h: V \rightarrow \mathbb{R}_{\geq 0}$. In motion planning, we can use, e.g., the Euclidean distance to the goal (as the crow flies).

Greedy (Best-First) Search

- UCS explores paths in all directions, with no bias towards
- What if we try to get “closer” to the goal?
- We need a measure of distance to the goal. It would be the length of the shortest path... but this is exactly what we want to compute!
- We can estimate the distance to the goal through a heuristic function $h: V \rightarrow \mathbb{R}_{\geq 0}$. In motion planning, we can use, e.g., the Euclidean distance to the goal (as the crow flies).
- A reasonable strategy is to always try to move in such a way that the estimated distance to the goal decreases: this is the basic idea of Greedy (Best-First) Search.

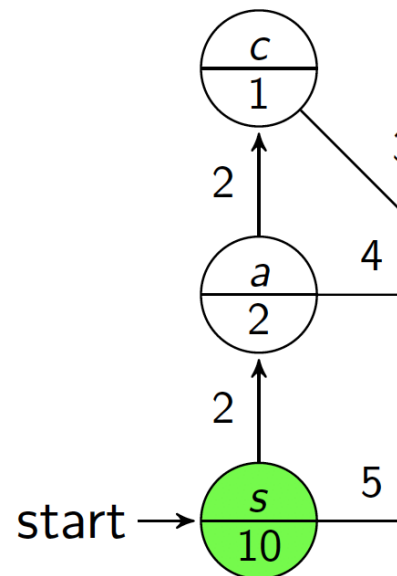
Greedy (Best-First) Search

```
Q ← ⟨start⟩;           // Initialize the queue with the start node
while Q is not empty do
    | Pick the path  $P$  with minimum heuristic cost  $h(\text{head}(P))$  from Q
    | if  $\text{head}(P) = \text{goal}$  then return  $P$ ;           // We have reached the goal
    | foreach vertex  $v$  such that  $(\text{head}(P), v) \in E$ , do
    |   | add  $\langle v, P \rangle$  to the queue Q;
return FAILURE ;           // Nothing left in the queue
```

Example of Greedy (Best-First) Search

Q:

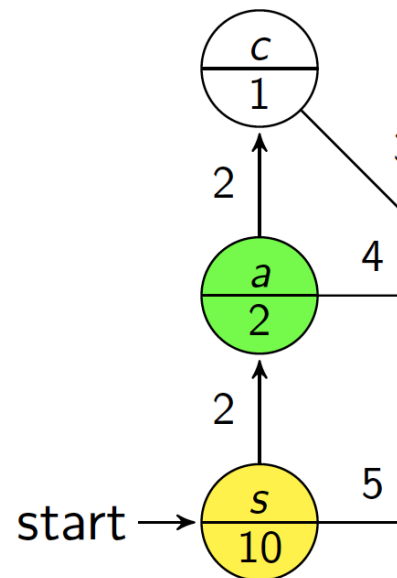
path	cost	h
$\langle s \rangle$	0	10



Example of Greedy (Best-First) Search

Q:

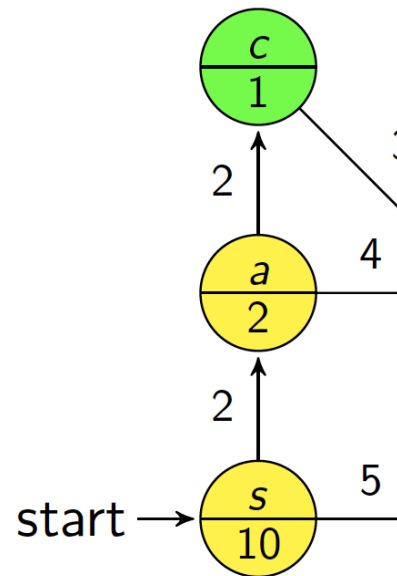
path	cost	h
$\langle a, s \rangle$	2	2
$\langle b, s \rangle$	5	3



Example of Greedy (Best-First) Search

Q:

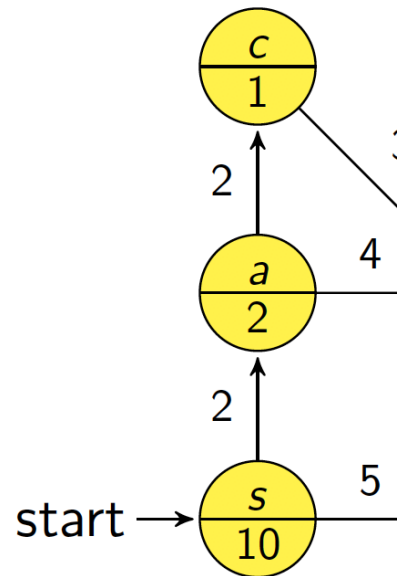
path	cost	h
$\langle c, a, s \rangle$	4	1
$\langle b, s \rangle$	5	3
$\langle d, a, s \rangle$	6	4



Example of Greedy (Best-First) Search

Q:

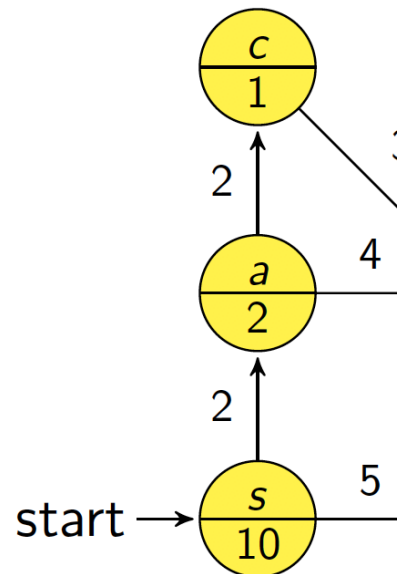
path	cost	h
$\langle b, s \rangle$	5	3
$\langle d, a, s \rangle$	6	4
$\langle d, c, a, s \rangle$	7	4



Example of Greedy (Best-First) Search

Q:

path	cost	h
$\langle g, b, s \rangle$	10	0
$\langle d, a, s \rangle$	6	4
$\langle d, c, a, s \rangle$	7	4



Remarks on Greedy (Best-First) Search

- Greedy (Best-First) search is similar in spirit to Depth-First Search, but it **keeps exploring until it has to back up** due to a dead end.

Remarks on Greedy (Best-First) Search

- Greedy (Best-First) search is similar in spirit to Depth-First Search, but it **keeps exploring until it has to back up** due to a dead end.
- Greedy search is **not complete** and **not optimal**, but it is often efficient, depending on the heuristic function h .

Remarks on Greedy (Best-First) Search

- Greedy (Best-First) search is similar in spirit to Depth-First Search, but it **keeps exploring until it has to back up** due to a dead end.
- Greedy search is **not complete** and **not optimal**, but it is often efficient, depending on the heuristic function h .
- Worst-case time and space complexity $O(b^m)$.

The A Search algorithm

The problems

- Uniform-Cost search is optimal, but may wander around a lot before reaching the goal.
- Greedy search is not optimal, but in some cases it is efficient. It is biased towards moving towards the goal. The non-optimality comes from neglecting “the past.”

The A Search algorithm

The problems

- Uniform-Cost search is optimal, but may wander around a lot before reaching the goal.
- Greedy search is not optimal, but in some cases it is efficient. It is biased towards moving towards the goal. The non-optimality comes from neglecting “the past.”

The idea

- Keep track both of the cost of the partial path to get to a vertex v and of the heuristic function estimating the cost to reach the goal from vertex v , $h(v)$.
- In other words, choose as a “ranking” function the sum of the two costs:

$$f(v) = g(v) + h(v)$$

- $g(v)$: **cost-to-come** (from the start to v).
- $h(v)$: **cost-to-go** estimate (from v to the goal).
- $f(v)$: estimated **cost of the path** (from the start to v and then to the goal).

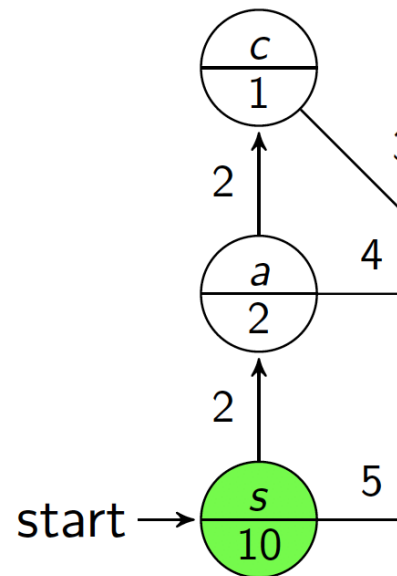
The A Search algorithm

```
Q ← ⟨start⟩;           // Initialize the queue with the
while Q is not empty do
    Pick the path  $P$  with minimum estimated cost  $f(P) = g(P)$ 
    from the queue  $Q$ ;
    if  $\text{head}(P) = \text{goal}$  then return  $P$  ;           // We have reached the goal
    foreach vertex  $v$  such that  $(\text{head}(P), v) \in E$ , do
        └ add  $\langle v, P \rangle$  to the queue  $Q$ ;
return FAILURE ;           // Nothing left in the queue
```

Example of A Search: Step 1

Q:

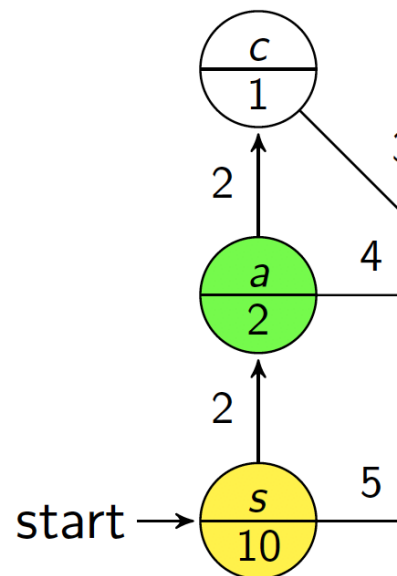
path	g	h	f
$\langle s \rangle$	0	10	10



Example of A Search: Step 2

Q:

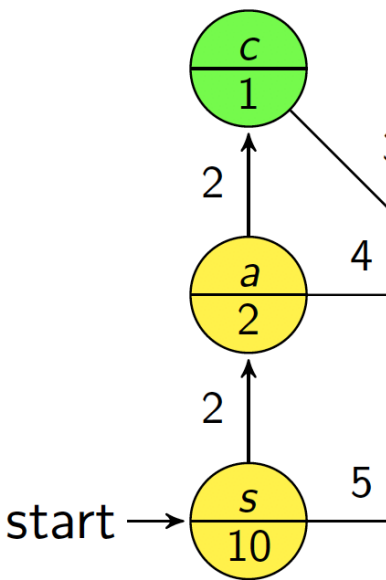
path	g	h	f
$\langle a, s \rangle$	2	2	4
$\langle b, s \rangle$	5	3	8



Example of A Search: Step 3

Q:

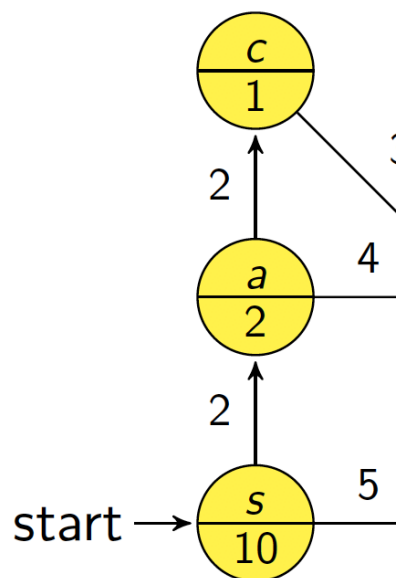
path	g	h	f
$\langle c, a, s \rangle$	4	1	5
$\langle b, s \rangle$	5	3	8
$\langle d, a, s \rangle$	6	5	11



Example of A Search: Step 4

Q:

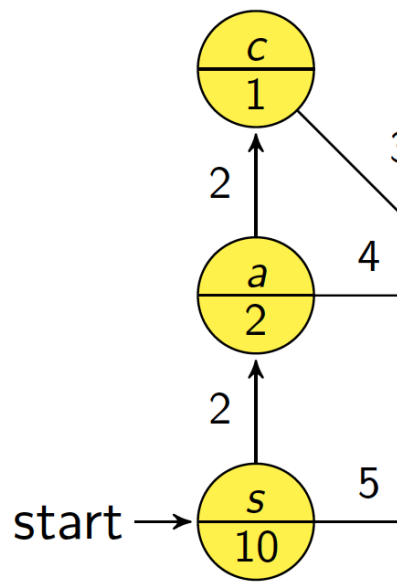
path	g	h	f
$\langle b, s \rangle$	5	3	8
$\langle d, a, s \rangle$	6	5	11
$\langle d, c, a, s \rangle$	7	5	12



Example of A Search: Step 5

Q:

path	g	h	f
$\langle g, b, s \rangle$	10	0	10
$\langle d, a, s \rangle$	6	5	11
$\langle d, c, a, s \rangle$	7	5	12



Remarks on the A Search algorithm

- A search is similar to UCS, with a bias induced by the heuristic h . If $h = 0$, $A = UCS$.
- The A search is complete, but is not optimal. (Recall that if $h = 0$ then $A = UCS$, and hence optimal.)

A^* Search

- Choose an **admissible** heuristic, i.e., such that $h(v) \leq c(v, w)$ for all v, w . (The star means “optimal.”)
- The A search with an admissible heuristic is called A^* search and is guaranteed to be optimal.

A* Search

A* Search

- In a minimization problem, an admissible heuristic $h(n)$ never overestimates the real cost.

A* Search

- In a minimization problem, an admissible heuristic $h(n)$ never overestimates the real cost.
- In a maximization problem, $h(n)$ is admissible if it never underestimates the real value.

A* Search

- In a minimization problem, an admissible heuristic $h(n)$ never overestimates the real cost.
- In a maximization problem, $h(n)$ is admissible if it never underestimates the real value.
- So admissible heuristics are optimistic.

A* Search

- In a minimization problem, an admissible heuristic $h(n)$ never overestimates the real cost.
- In a maximization problem, $h(n)$ is admissible if it never underestimates the real value.
- So admissible heuristics are optimistic.
- Best-first search using $f(n) = g(n) + h(n)$ and an admissible heuristic is as A* search.

A* Search

- In a minimization problem, an admissible heuristic $h(n)$ never overestimates the real cost.
- In a maximization problem, $h(n)$ is admissible if it never underestimates the real value.
- So admissible heuristics are optimistic.
- Best-first search using $f(n) = g(n) + h(n)$ and an admissible heuristic is as A* search.
- A* tree search is complete & optimal.

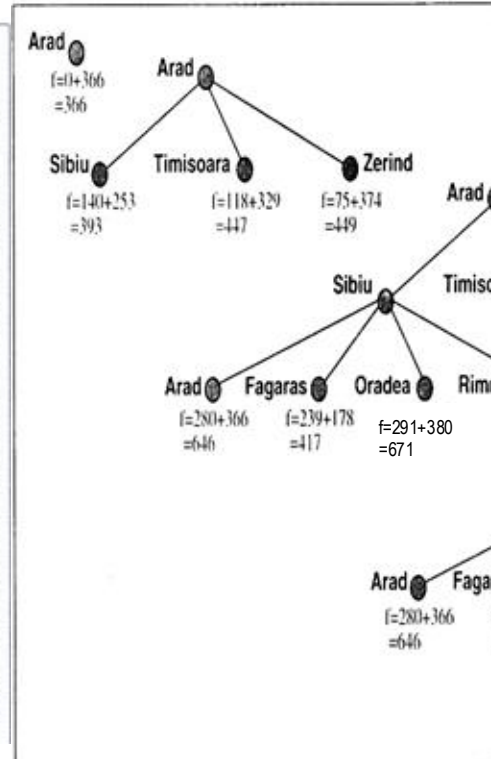
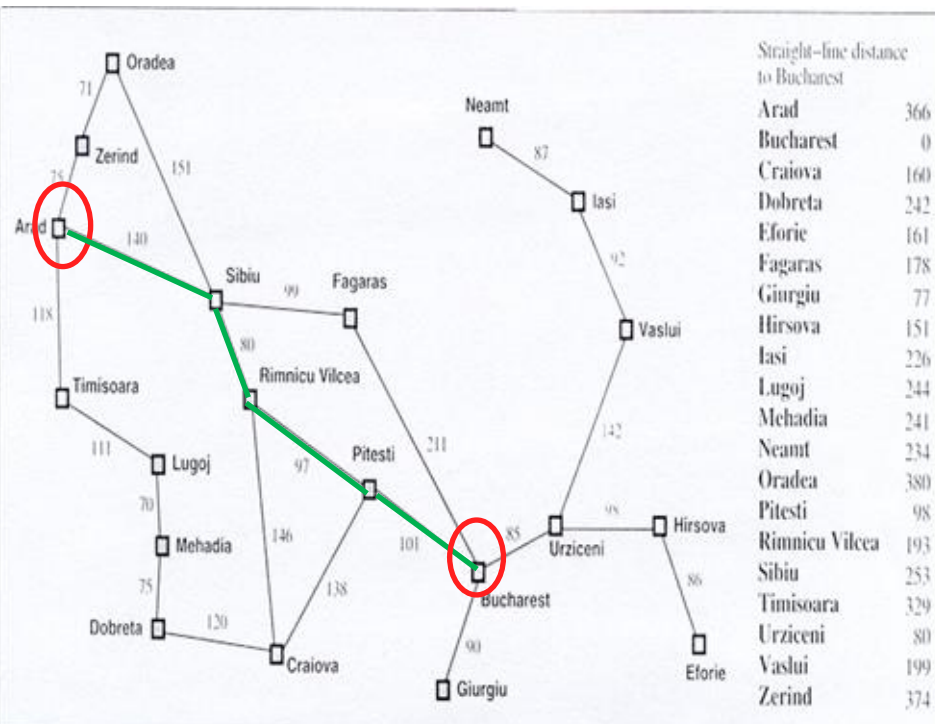
A* Search

function A*-SEARCH (problem) returns a solution or failure
 return BEST-FIRST-SEARCH (problem, g+h)

A* Search

function A*-SEARCH (problem) returns a solution or failure
 return BEST-FIRST-SEARCH (problem, $g+h$)

$f(n)$ = estimated cost of the cheapest solution through
 $= g(n) + h(n)$
 $=$ path cost + heuristic estimate of distance to goal



A* Search

- Idea: avoid expanding paths that are already expensive
- Evaluation function $f(n) = g(n) + h(n)$
- $g(n)$ = cost so far to reach n
- $h(n)$ = estimated cost from n to goal
- $f(n)$ = estimated total cost of path through n to goal
- Best First search has $f(n) = h(n)$
- Uniform Cost search has $f(n) = g(n)$

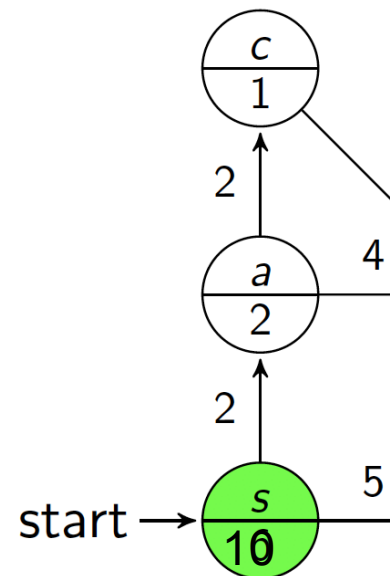
Admissible Heuristics

- A heuristic $h(n)$ is **admissible** if for every node n , where $h^*(n)$ is the **true** cost to reach the goal state from n , $h(n) \leq h^*(n)$.
- An admissible heuristic **never overestimates** the cost to reach the goal, i.e., it is **optimistic**.
- Example: $h_{SLD}(n)$ (never overestimates the actual distance).
- **Theorem:** If $h(n)$ is admissible, A^* using TREE-SEARCH is optimal.

Example of A* Search: Step 1

Q:

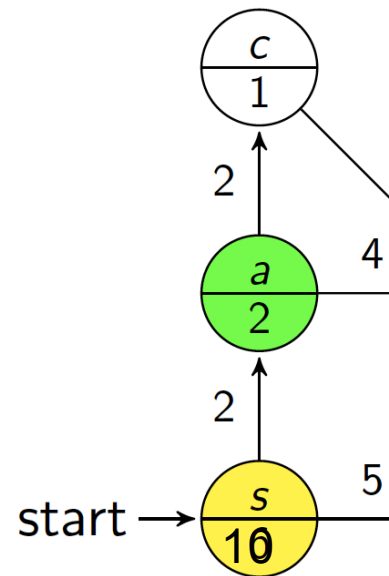
path	g	h	f
$\langle s \rangle$	0	10	10



Example of A* Search: Step 2

Q:

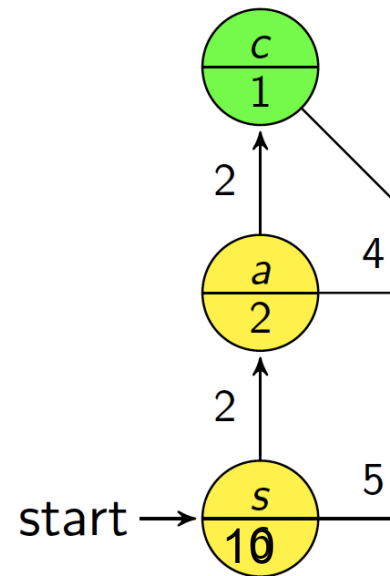
path	g	h	f
$\langle a, s \rangle$	2	2	4
$\langle b, s \rangle$	5	3	8



Example of A* Search: Step 3

Q:

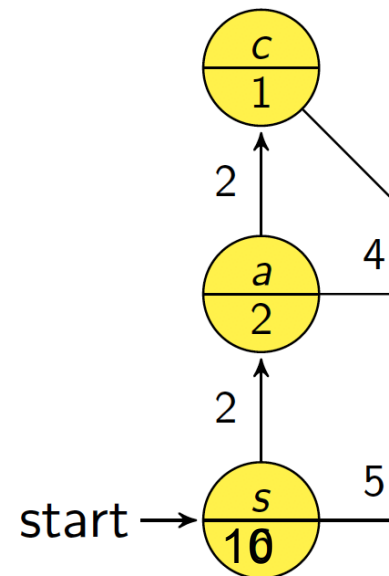
path	g	h	f
$\langle c, a, s \rangle$	4	1	5
$\langle d, a, s \rangle$	6	1	7
$\langle b, s \rangle$	5	3	8



Example of A* Search: Step 4

Q:

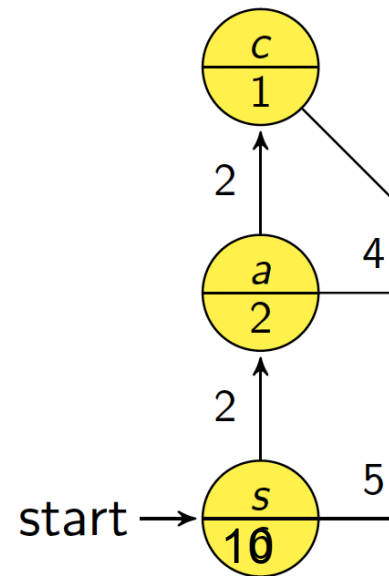
path	g	h	f
$\langle d, a, s \rangle$	6	1	7
$\langle b, s \rangle$	5	3	8
$\langle d, c, a, s \rangle$	7	1	8



Example of A* Search: Step 5

Q:

path	g	h	f
$\langle g, d, a, s \rangle$	8	0	8
$\langle b, s \rangle$	5	3	8
$\langle d, c, a, s \rangle$	7	1	8



Properties of A*

- **Complete?** Yes
(unless there are infinitely many nodes with $f \leq f(G)$, i.e. $f(G)$ is not a lower bound)
- **Time/Space?** Exponential: b^d
except if: $|h(n) - h^*(n)| \leq O(\log h^*(n))$
- **Optimal?** Yes
- **Optimally Efficient:** Yes
(No algorithm with the same heuristic is guaranteed to visit fewer nodes)

Completeness of A*

- Because A* expands nodes in order of increasing f , it will eventually expand to reach a goal state. This is true unless there are nodes with $f(n) \leq f^*$.
- How could this happen?
 - There is a node with an infinity branching factor.
 - There is a path with finite path cost but an infinite cost on it.
- So, A* is complete on graphs with a finite branching factor b and some positive constant ϵ such that every operator costs at least ϵ .

How to generate h-function

Heuristics ($h(n)$) for A*

A typical instance of the 8-puzzle

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

Heuristics?

h_1 : #tiles in wrong position

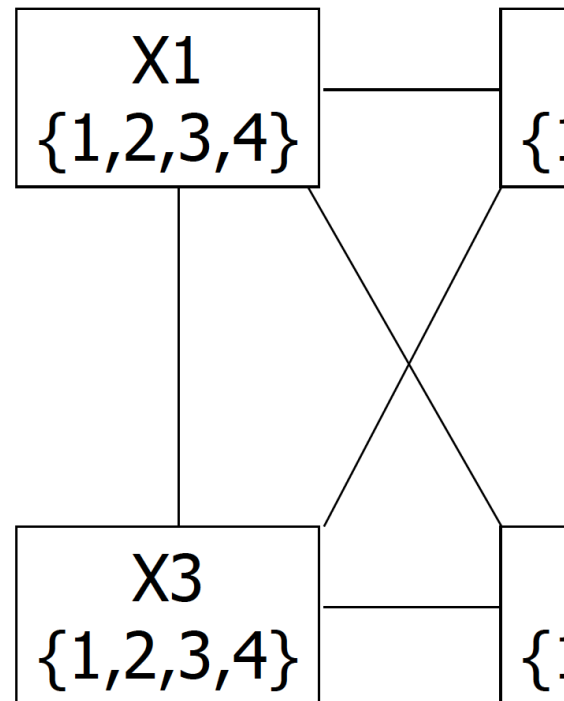
h_2 : sum of Manhattan distances of the tiles from their goal

h_2 dominates h_1 : $\forall n, h_2(n) \geq h_1(n)$

Constraint Satisfaction Problem

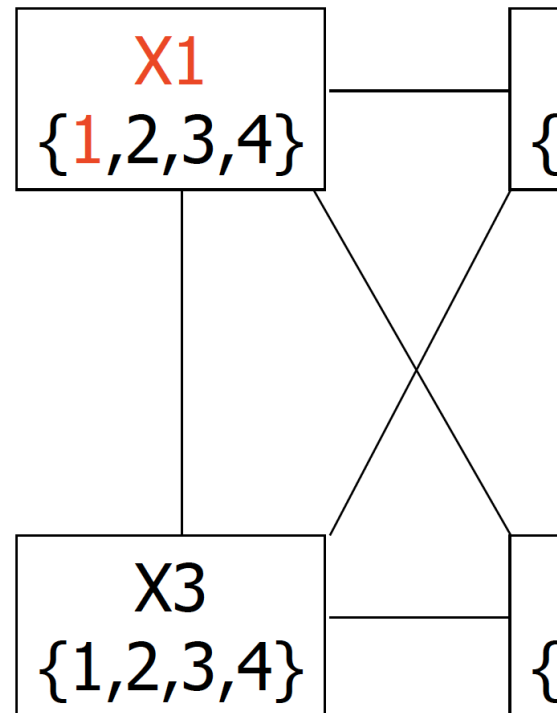
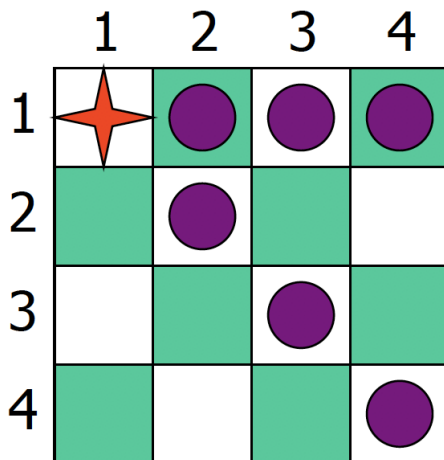
Example: 4-Queens Problem

	X1	X2	X3	X4
1				
2				
3				
4				

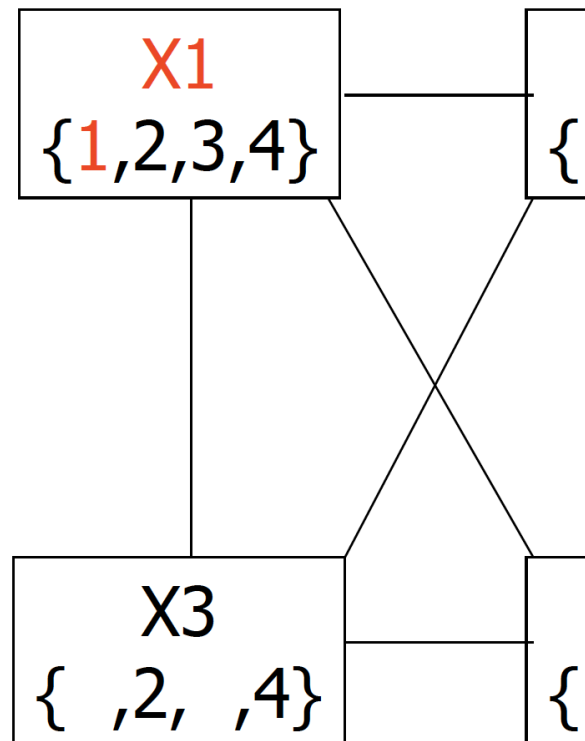
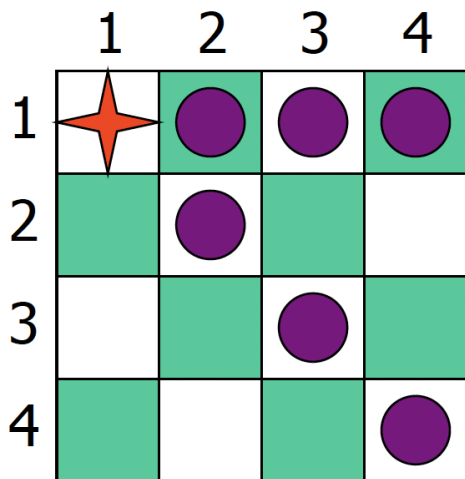


(From Bonnie Dorr, U of Md, CMSC 421)

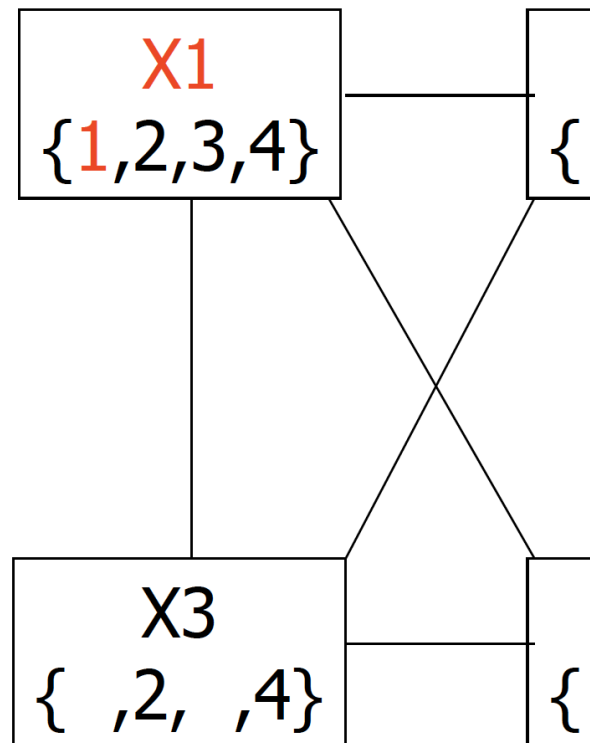
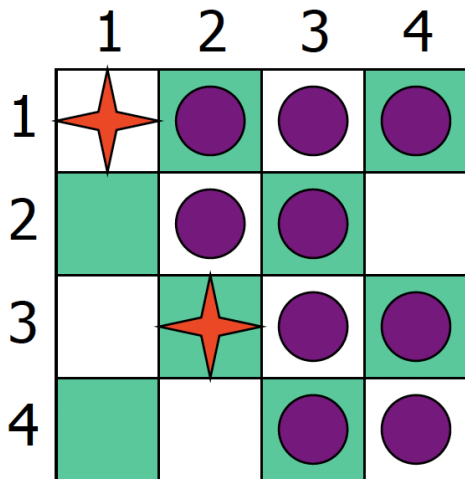
Example: 4-Queens Problem



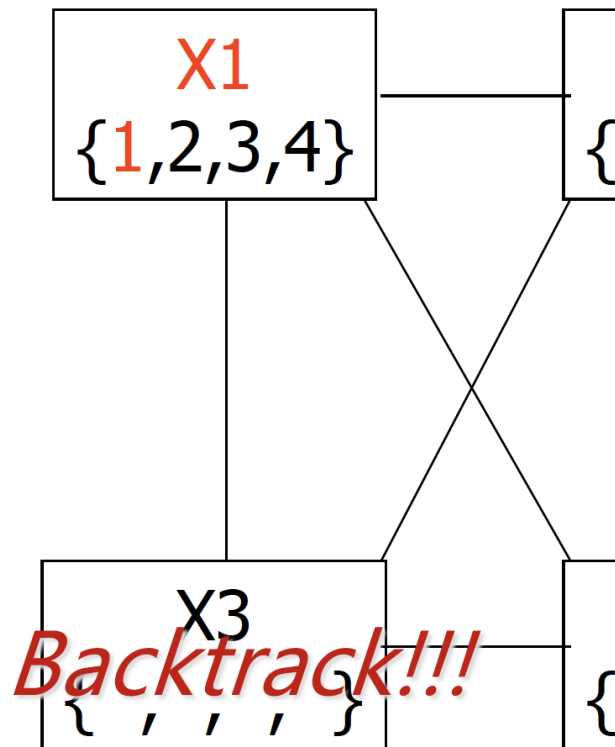
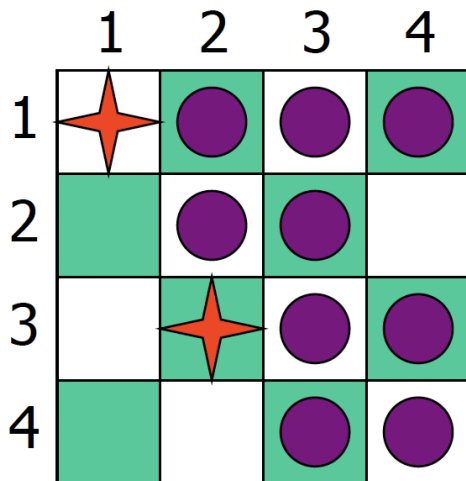
Example: 4-Queens Problem



Example: 4-Queens Problem

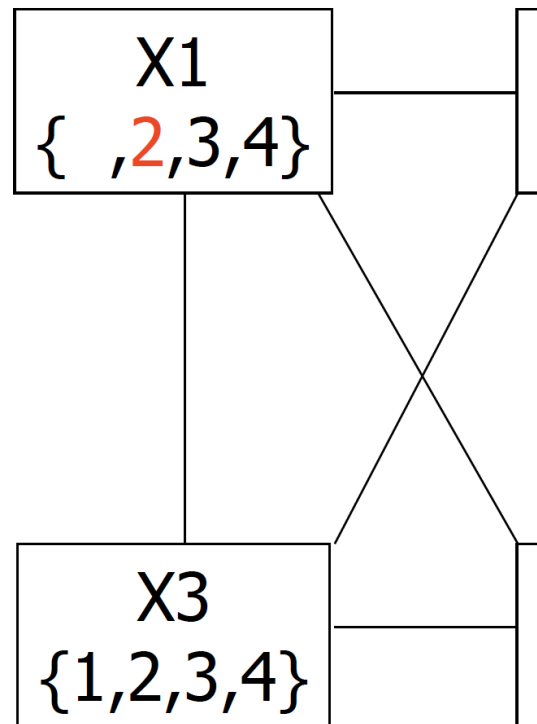
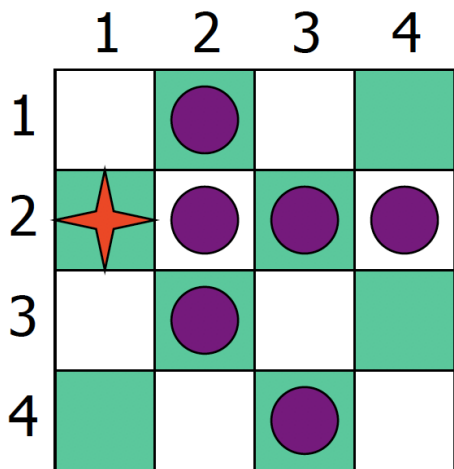


Example: 4-Queens Problem

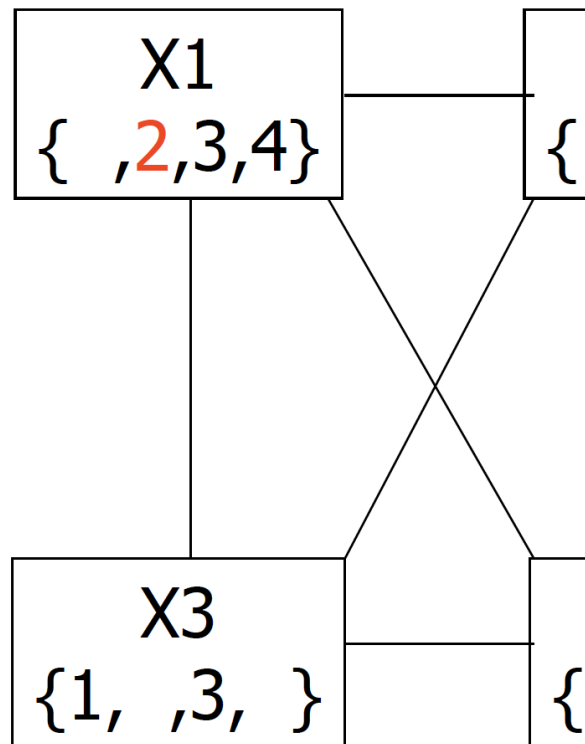
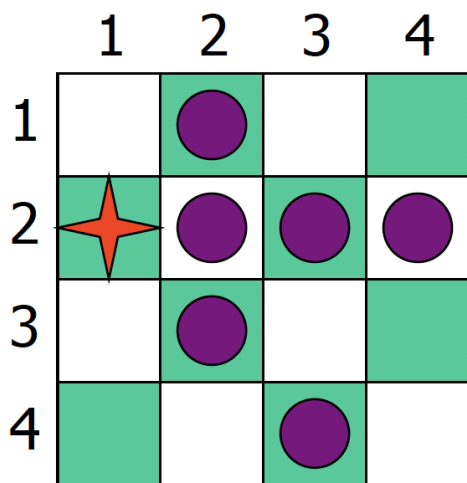


Example: 4-Queens Problem

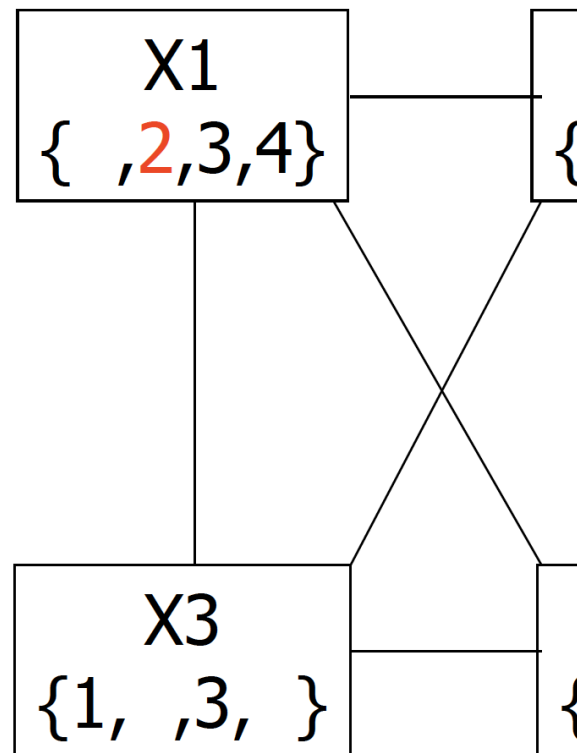
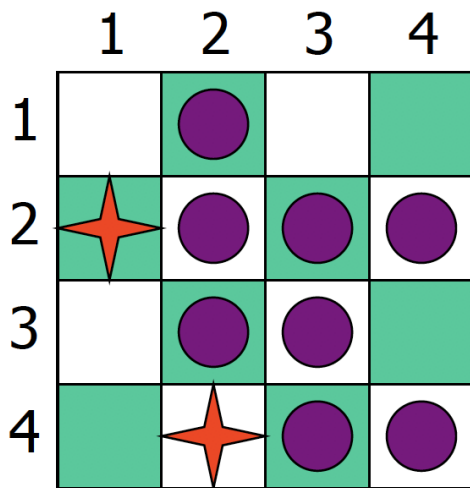
Picking up a little later after two steps of backtracking....



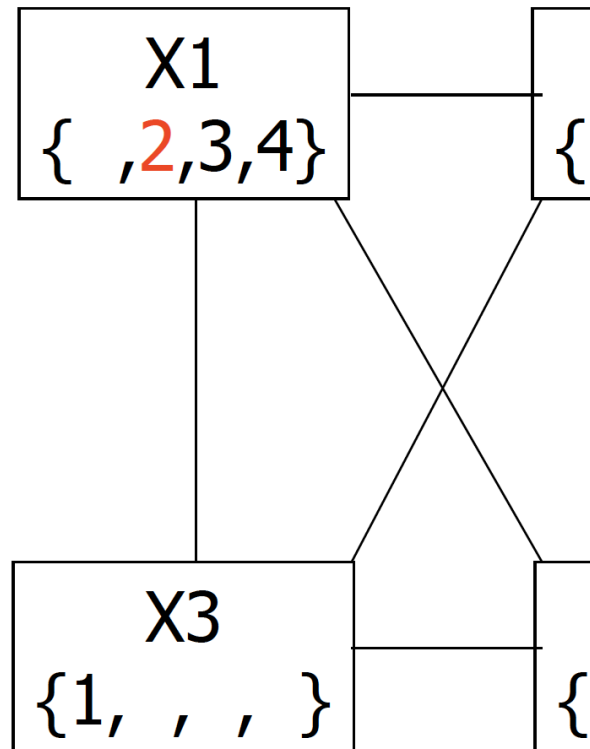
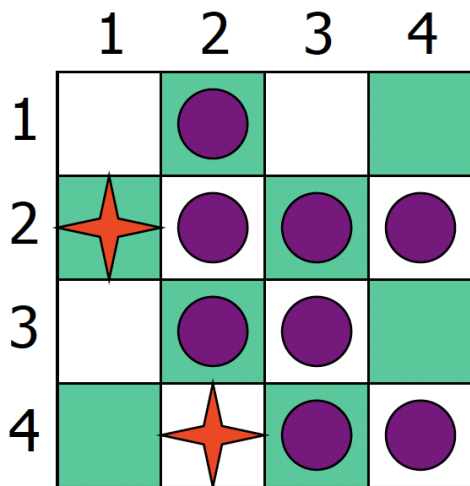
Example: 4-Queens Problem



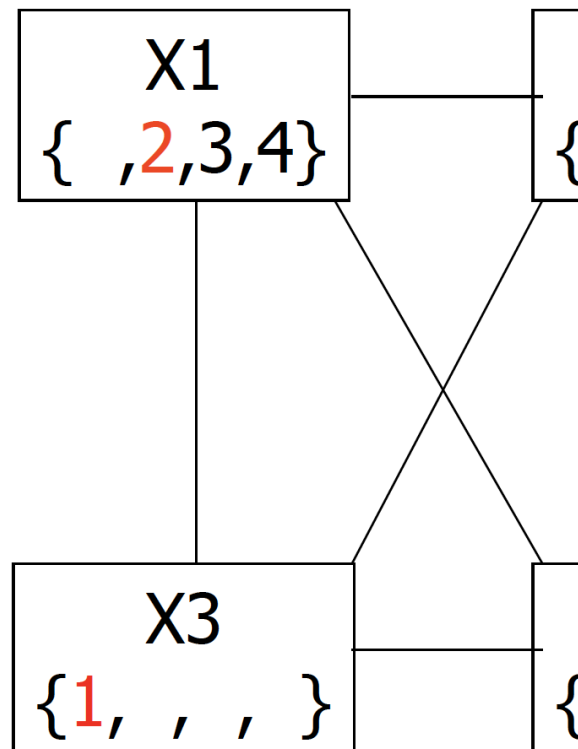
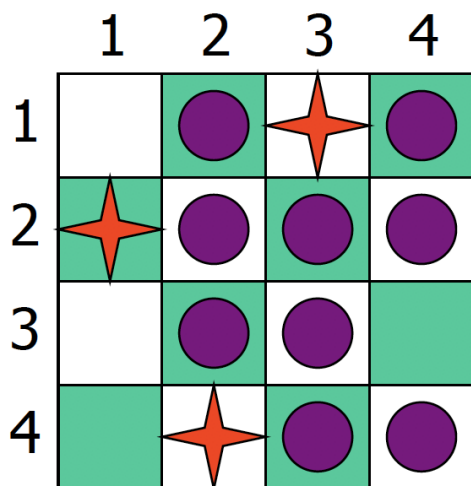
Example: 4-Queens Problem



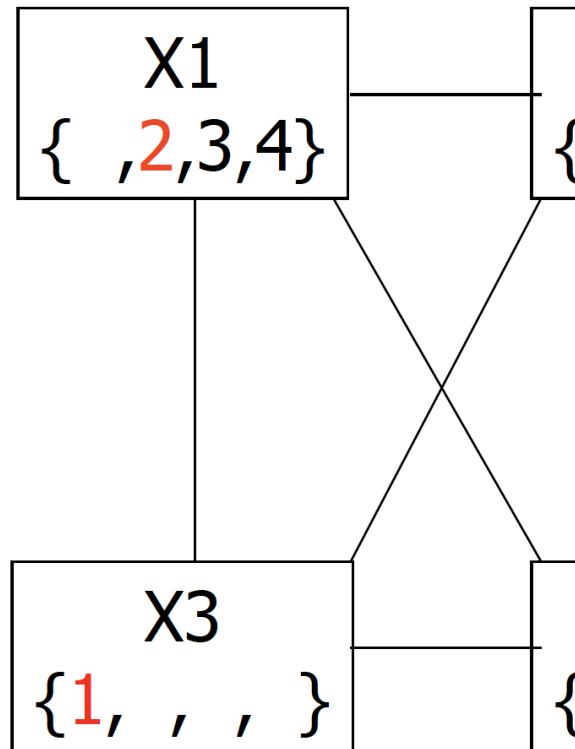
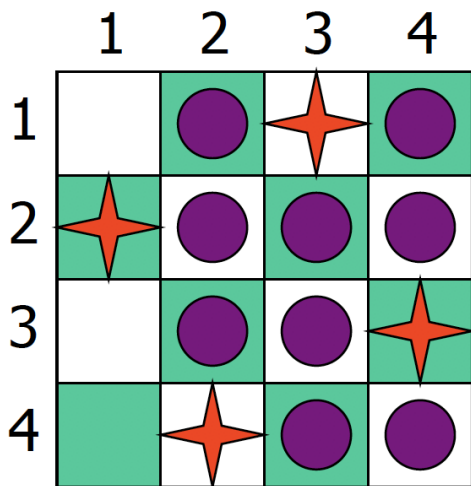
Example: 4-Queens Problem



Example: 4-Queens Problem



Example: 4-Queens Problem



Standard Search vs. Constraint satisfaction problems (CSP)

- First studied general state space search in isolation
 - Standard search problem: search in a state space

Standard Search vs. Constraint satisfaction problems (CSP)

- First studied general state space search in isolation
 - Standard search problem: search in a state space
- Standard search problem: **state** is a "black box" structure that supports successor function and goal test

Standard Search vs. Constraint satisfaction problems (CSP)

- First studied general state space search in isolation
 - Standard search problem: search in a state space
- Standard search problem: **state** is a "black box" structure that supports successor function and goal test
- CSP:
 - **state** is defined by **variables** X_i with **values** from D_i
 - **goal test** is a set of **constraints** specifying allowed combinations of values for subsets of variables

Standard Search vs. Constraint satisfaction problems (CSP)

- First studied general state space search in isolation
 - Standard search problem: search in a state space
- Standard search problem: **state** is a "black box" structure that supports successor function and goal test
- CSP:
 - **state** is defined by **variables** X_i with **values** from D_i
 - **goal test** is a set of **constraints** specifying allowed combinations of values for subsets of variables
- Simple example of a **formal representation language**
- Allows useful **general-purpose** algorithms with more powerful than standard search algorithms

Probabilistic Reasoning: Probability Model

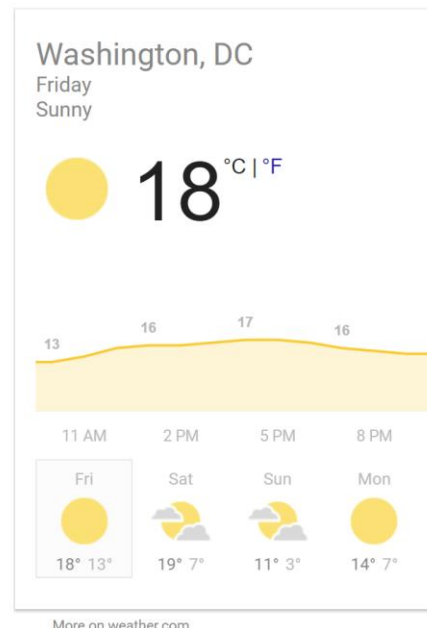
Recap

- Search & Satisfiability in discrete space
- Basic optimization in continuous and discrete space
- Deterministic / Symbolic Reasoning
- Known environment
- Full observability
- Deterministic change

This section

- Reason about uncertainty
 - Signature of real-world scenarios
 - Partial observability

Is it raining



3/19/2026

This section

- Reason about uncertainty
 - Signature of real-world scenarios
 - Partial observability
 - Sometimes, introduce uncertainty to model to abstract how the world really is
 - Need probabilistic models for knowledge representation & reasoning
 - Provide a way of summarizing the uncertainty comes from our laziness and ignorance

- Probability Model and Probabilistic Inference
- Chain Rule, Independence, Bayes

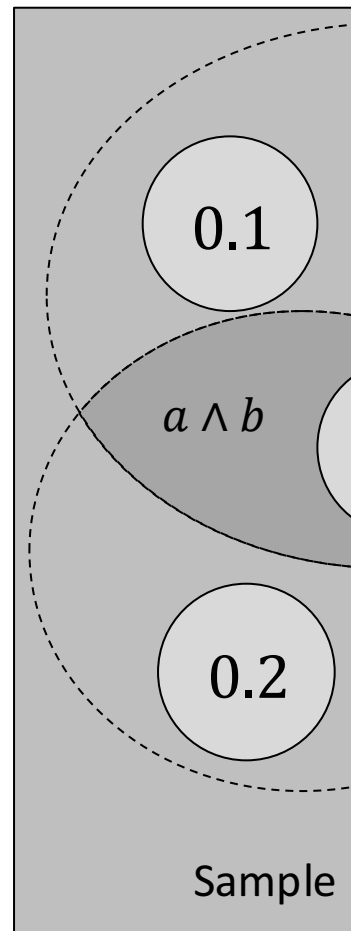
Basic Rules

- Negation relationship

$$P(a) = 1 - P(\neg a)$$

- Inclusion-exclusion principle

$$P(a \vee b) = P(a) + P(b) - P(a \wedge b)$$



Basic Rules

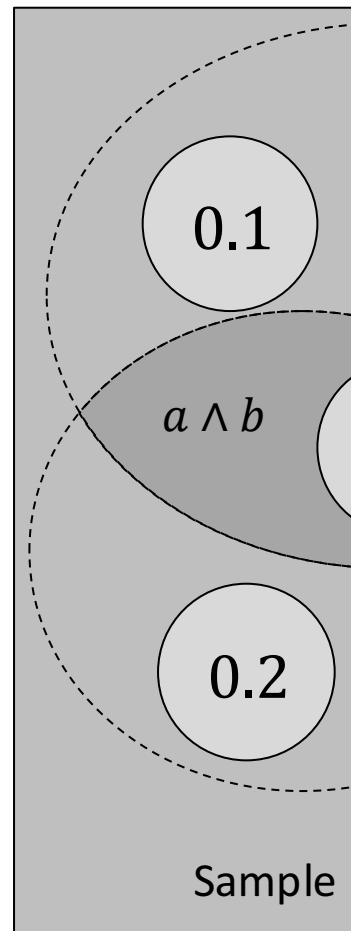
- Negation relationship

$$P(a) = 1 - P(\neg a)$$

Clearly $P(a) + P(1 - a) = 1$

- Inclusion-exclusion principle

$$P(a \vee b) = P(a) + P(b) - P(a \wedge b)$$



Basic Rules

- Product rule

$$P(a \wedge b) = P(a|b)P(b)$$

- Sum rule

$$P(a) = \sum_k P(a \wedge b_k)$$

- If b_k 's are mutually exclusive and exhaustive

Basic Rules

- Product rule

$$P(a \wedge b) = P(a|b)P(b)$$

Derived directly from $P(a|b) = \frac{P(a \wedge b)}{P(b)}$

Similarly, $P(a \wedge b) = P(b|a)P(a)$

So $P(a|b)P(b) = P(a \wedge b) = P(b|a)P(a)$

- Sum rule

$$P(a) = \sum_k P(a \wedge b_k)$$

- If b_k 's are mutually exclusive and exhaustive

Joint Probability Distribution

				
	.05	.2	0	.1
	.1	0	.1	0
	0	.1	.05	.1
	.1	0	.1	0

Joint Probability Distribution

				
	.05	.2	0	.1
	.1	0	.1	0
	0	.1	.05	.1
	.1	0	.1	0

Random variables are
Color: {red, blue, yellow, black}
Type: {Sedan, SUV, Coupe, Truck}

Joint probability distribution
 $P(\text{Color}, \text{Type})$

Specifies $P(\text{Color} = c, \text{Type} = t)$
 $\forall c \in \text{domain}(\text{Color}), \forall t \in \text{domain}(\text{Type})$

$P(\text{Color} = \text{red}, \text{Type} = \text{Sedan}) = 0.05$

Compute Marginal Prob

- $P(\text{Color} = \text{red}) = ?$



	.05	.2
	.1	0
	0	.1
	.1	0

Compute Marginal Prob

- $P(\text{Color} = \text{red}) = ?$

- Recall sum rule

$$P(a) = \sum_k P(a \wedge b_k)$$

Where b_k 's are mutually exclusive and exhaustive

- Marginal probability



	.05	.2
	.1	0
	0	.1
	.1	0

$$P(\text{Color} = \text{red}) = \sum_{t \in \text{domain}(\text{Type})} P(\text{Color} = \text{red} \cap \text{Type} = t)$$

Marginal Probability Distr

				
	.05	.2	0	.1
	.1	0	.1	0
	0	.1	.05	.1
	.1	0	.1	0

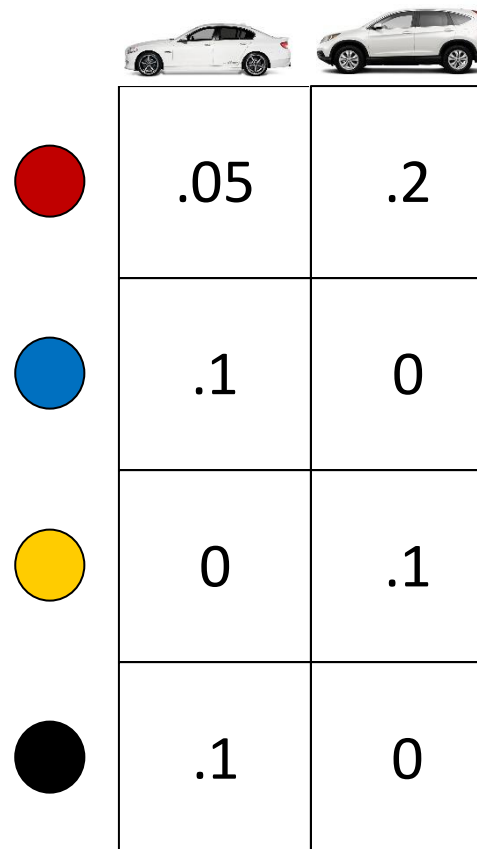
Marginal Probability Distr

- Sum up the row / column to get the marginal probability

		.25	.3	.25	.2
					
.35		.05	.2	0	.1
.2		.1	0	.1	0
.25		0	.1	.05	.1
.2		.1	0	.1	0

Compute Conditional Pro

- $P(\text{Color} = \text{red} | \text{Type} = \text{SUV}) = ?$
- Scale the joint probability according to marginal probability



		
	.05	.2
	.1	0
	0	.1
	.1	0

Compute Conditional Pro

- $P(\text{Color} = \text{red} | \text{Type} = \text{SUV}) = ?$
- Scale the joint probability according to marginal probability



.05

.2



.1

0



0

.1



.1

0

Recall $P(\phi|e) = \frac{P(\phi \wedge e)}{P(e)}$

So $P(\text{Color} = \text{red} | \text{Type} = \text{SUV}) = \frac{P(\text{Color} = \text{red} \wedge \text{Type} = \text{SUV})}{P(\text{Type} = \text{SUV})}$

$P(\text{Type} = \text{SUV})$

$= \sum_{c \in \text{domain}(\text{Color})} P(\text{Color} = c \cap \text{Type} = \text{SUV})$

$= 0.2 + 0.1 = 0.3$

$P(\text{Color} = \text{red} | \text{Type} = \text{SUV}) = \frac{0.05}{0.3}$

Independence

- Definition: Random variable X and Y are independent iff
 - $\mathbf{P}(X|Y) = \mathbf{P}(X)$
 - or $\mathbf{P}(Y|X) = \mathbf{P}(Y)$
 - or $\mathbf{P}(X, Y) = \mathbf{P}(X)\mathbf{P}(Y)$
- Random variables $X_1, \dots, X_n$ are independent iff
 - $\mathbf{P}(X_1, X_2, \dots, X_n) = \mathbf{P}(X_1)\mathbf{P}(X_2) \dots \mathbf{P}(X_n)$

Conditional Independence

- Definition: X_1 and X_2 are conditionally independent given X_3 iff $\mathbf{P}(X_1|X_2, X_3) = \mathbf{P}(X_1|X_3)$
 - Once the value of X_3 is known, knowing X_2 doesn't tell anything more about X_1
 - Equivalent definition: $\mathbf{P}(X_1, X_2 | X_3) = \mathbf{P}(X_1 | X_3) \mathbf{P}(X_2 | X_3)$
 - In a sense, the dependence between X_1 and X_2 “dissolves” once the knowledge of X_3 is made available

Bayes' Rule

- Recall product rule $P(a \wedge b) = P(a)P(b|a)$
- Bayes' Rule $P(b|a) = \frac{P(a|b)P(b)}{P(a)}$

Bayes' Rule

- Recall product rule $P(a \wedge b) = P(a|b)P(b)$

Similarly, $P(a \wedge b) = P(b|a)P(a)$

So $P(a|b)P(b) = P(a \wedge b) = P(b|a)P(a)$

- Bayes' Rule $P(b|a) = \frac{P(a|b)P(b)}{P(a)}$

Bayes' Rule

- $$\mathbf{P}(Y|X) = \frac{\mathbf{P}(X|Y)\mathbf{P}(Y)}{\mathbf{P}(X)}$$

Bayes' Rule

- $\mathbf{P}(Y|X) = \frac{\mathbf{P}(X|Y)\mathbf{P}(Y)}{\mathbf{P}(X)}$

$$P(Y = y|X = x) = \frac{P(X = x|Y = y)P(Y = y)}{P(X = x)}$$

Marginal Probability

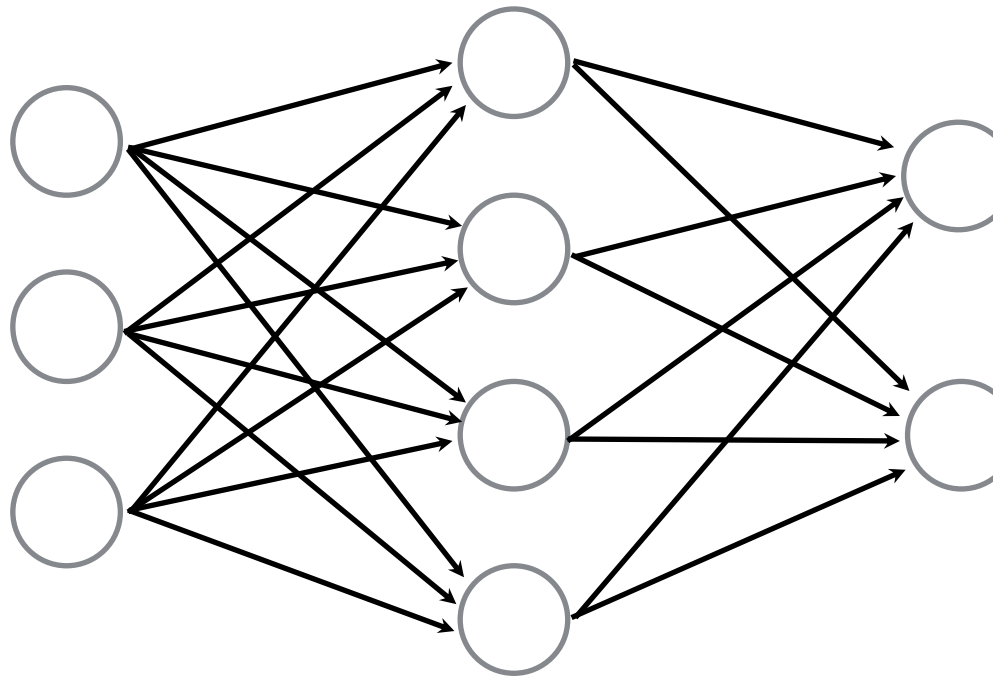
Product Rule

$$P(X = x) = \sum_{y' \in Y} P(X = x, Y = y') = \sum_{y' \in Y} P(X = x|Y = y')$$

$$P(Y = y|X = x) = \frac{P(X = x|Y = y)P(Y = y)}{\sum_{y' \in Y} P(X = x|Y = y')P(Y = y)}$$

Neural Network

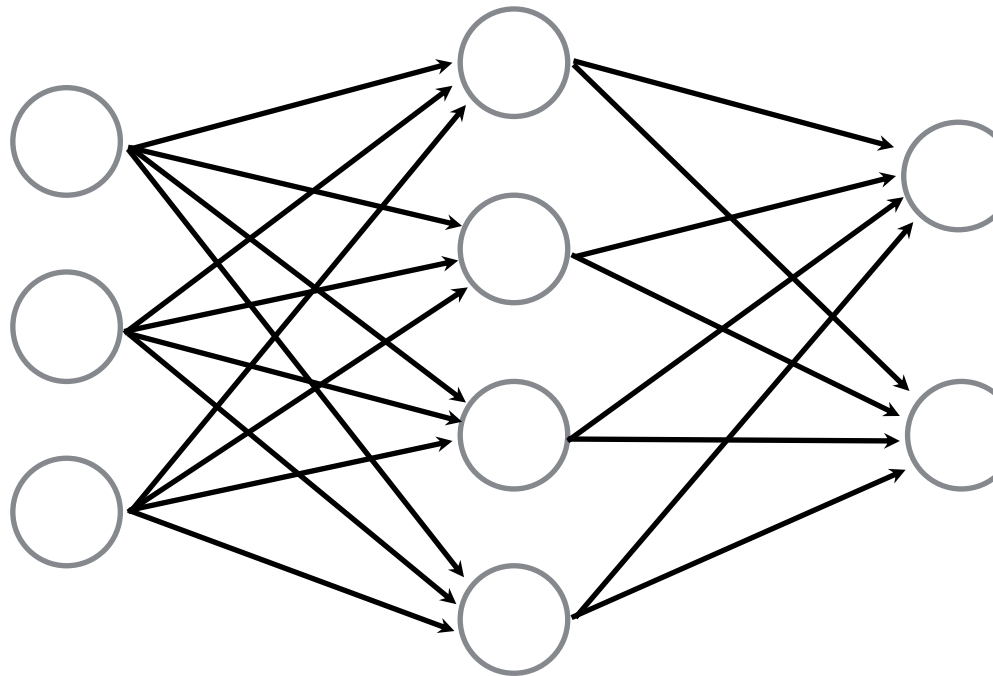
a collection of connected perceptrons



Connect a bunch of perceptrons together ...

Neural Network

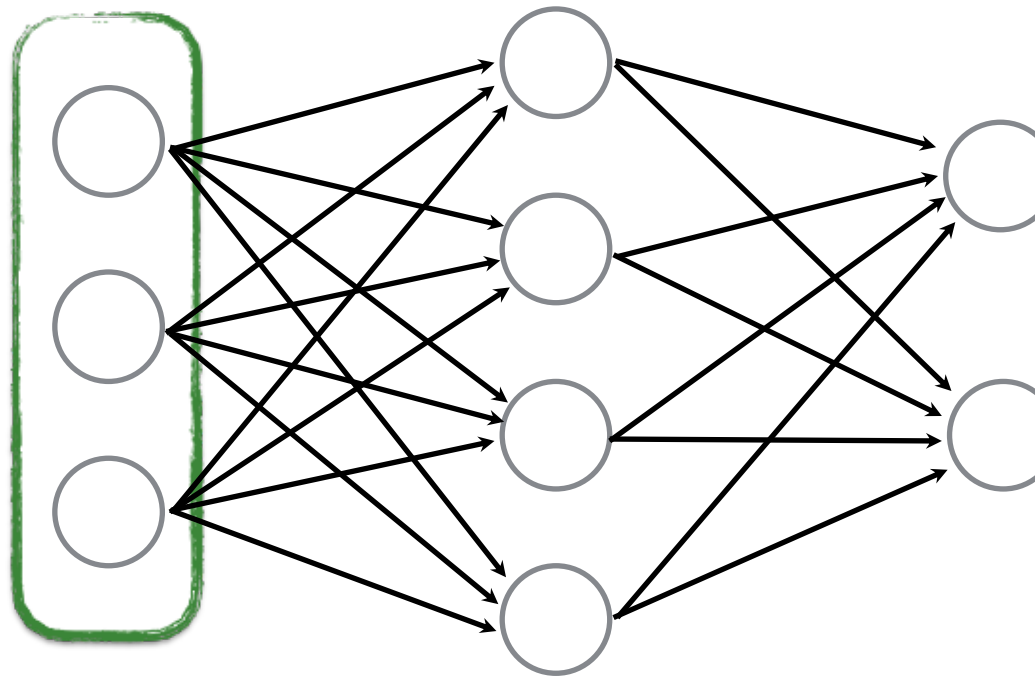
a collection of connected perceptrons



How many perceptrons in this neural network?

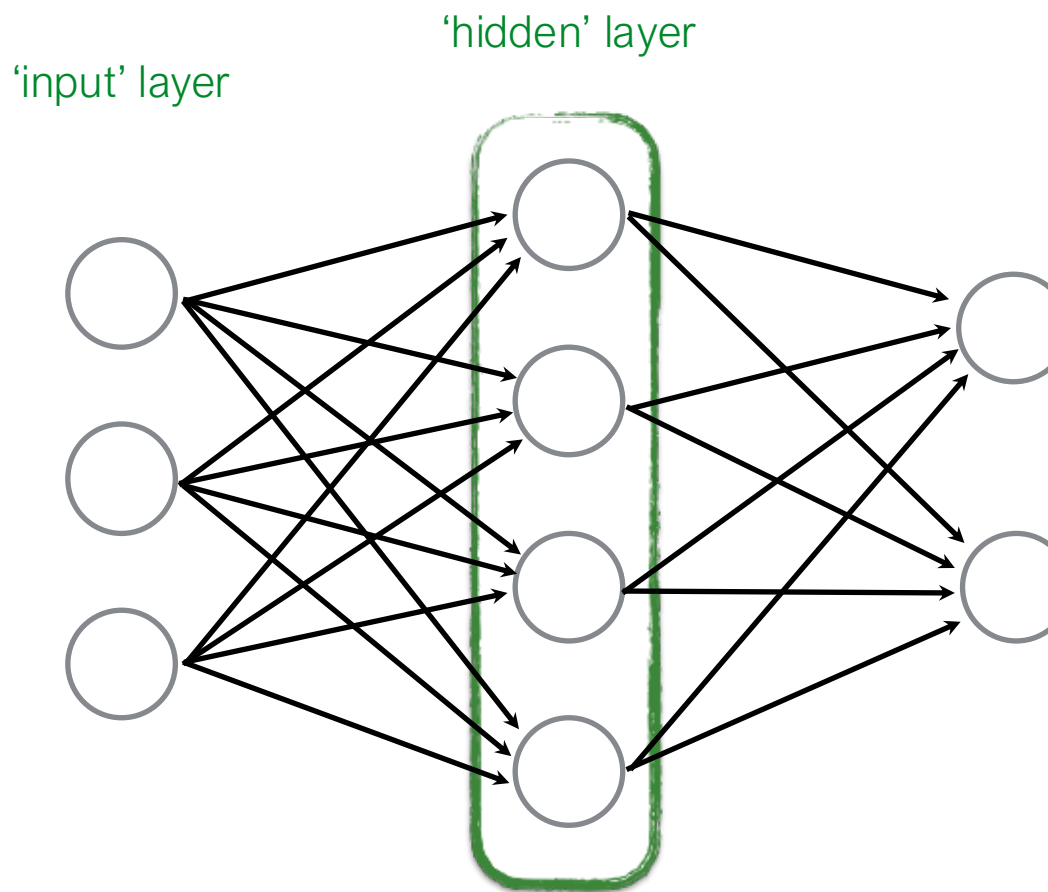
Some terminology...

'input' layer



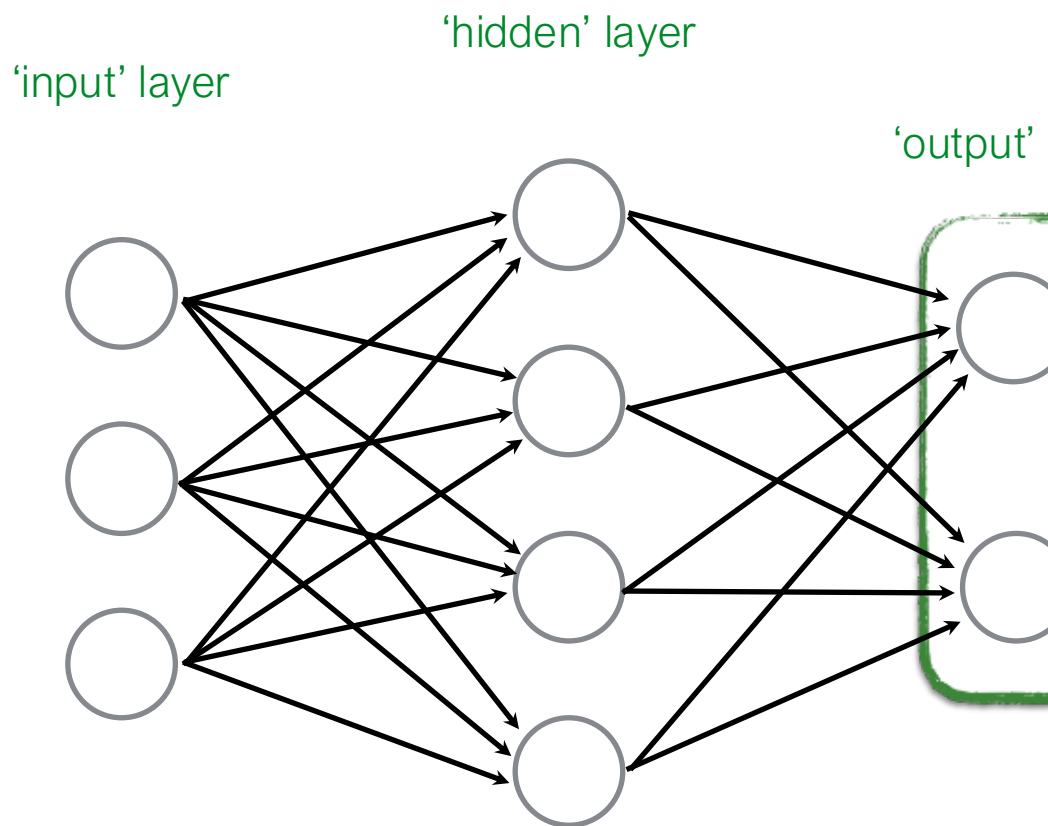
...also called a **Multi-layer**

Some terminology...



...also called a **Multi-layer**

Some terminology...



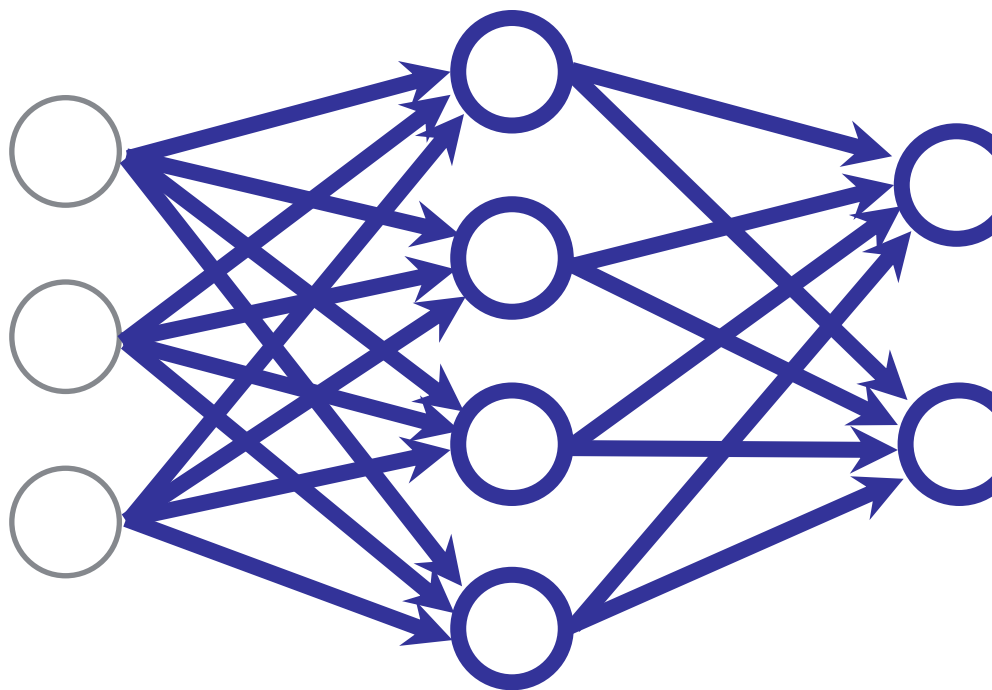
...also called a **Multi-layer**

How many neurons (perceptrons)?

4 + 2

How many weights (edges)?

$(3 \times 4) + (4 \times 2)$

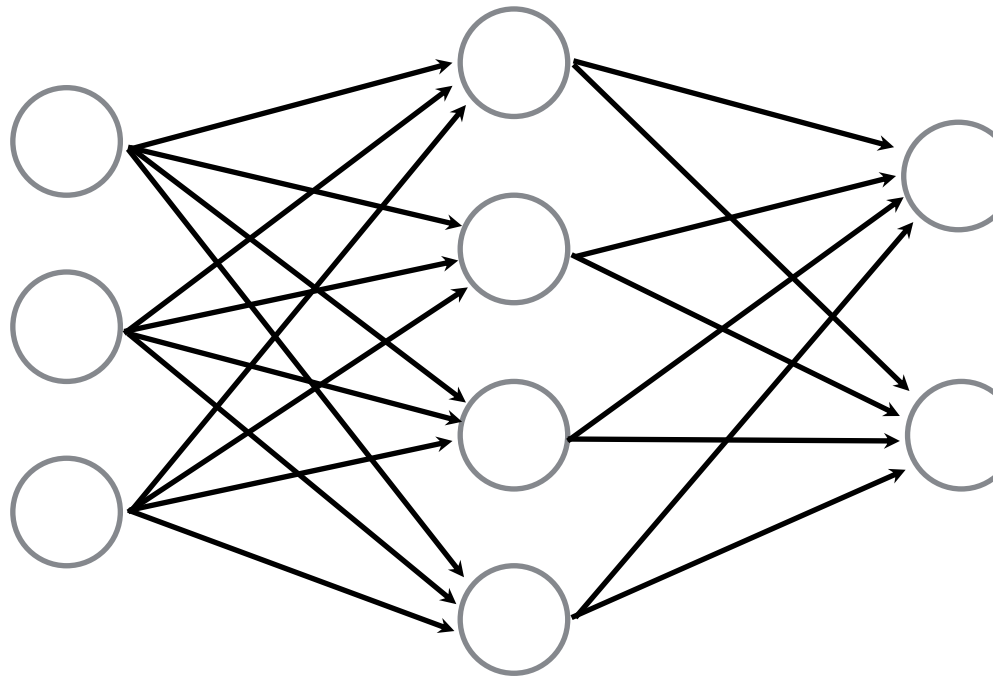


How many learnable parameters total?

20 + 4

bias

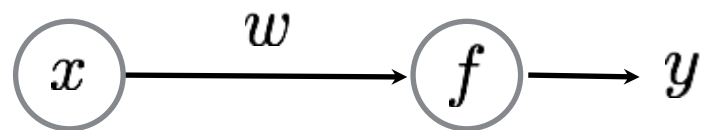
performance usually tops out at 2-3 layers,
deeper networks don't really improve performance.



...with the exception of **convolutional** networks for im

Training perceptron

world's smallest percep



$$y = wx$$

(a.k.a. line equation, linear regression)

Learning a Perceptron

Given a set of samples and a Perceptron

$$\{x_i, y_i\}$$

$$y = f_{\text{PER}}(x; w)$$

Estimate the parameters of the Perceptron

$$w$$